



Cluster Autoscaler in Kubernetes

Understanding Kubernetes for Orchestration

Introduction

Cluster Autoscaler (CA) automatically scales the **number of worker nodes** in a Kubernetes cluster based on demand.

This guide will cover:

- **Why Cluster Autoscaler is needed?**
 - **How CA works?**
 - **Deploying Cluster Autoscaler in EKS**
 - **Common issues and FAQs**
-

Why Use Cluster Autoscaler?

- **Dynamically adds or removes worker nodes** based on resource demand.
 - **Ensures applications run smoothly** even under high load.
 - **Reduces cloud costs** by automatically removing unused nodes.
-

Step 1: Deploy Cluster Autoscaler in EKS

1. **Deploy Cluster Autoscaler using Helm:**

```
helm repo add autoscaler https://kubernetes.github.io/autoscaler  
helm install cluster-autoscaler autoscaler/cluster-autoscaler
```

2. **Enable Auto Scaling in EKS:**

```
eksctl scale nodegroup --cluster my-cluster --name my-nodegroup  
--nodes-min=2 --nodes-max=5
```



3. Verify Cluster Autoscaler:

```
kubectl get pods -n kube-system | grep cluster-autoscaler
```

Common Issues and Troubleshooting

Problem	Solution
Cluster not scaling	Ensure nodegroup has auto-scaling enabled
Insufficient CPU error	Check available node capacity
Pods stuck in Pending	Verify if new nodes are being added

Conclusion

Cluster Autoscaler ensures **efficient node resource management**, reducing costs while maintaining performance.

👉 Deploy **CA in your EKS cluster** and observe node scaling behavior!